Capstone Project Checklist (100 Marks)

Project Title: EventZen – Event Management System
Tech Stack:

- React (Frontend),
- Spring Boot / Spring + Node Microservices / Dotnet (Backend) or any other combination,
- JWT Authentication, Docker, GitHub/GitLab Repository

Concept	Sub Topic	Criteria	Grading
Application Architecture & Modules **20 Marks**	Functional Modules (Excluding User Module) **10 Marks**	Completeness of CRUD operations Business logic implementation Microservice integration (if implemented)	10 – Fully implemented & functional modules 7 – Mostly functional; minor missing features 4 – Partially implemented 0 – Not implemented properly
	User Module with Authentication **10 Marks**	User Registration & Login JWT-based authentication & authorization Role-based access (Admin/User) User management endpoints	10 – Fully working JWT & user management 7 – Working with minor issues 4 – Partially working 0 – Not implemented
Backend Development (APIs, Logic, Architecture) – **20 Marks**	API Endpoints Quality **10 Marks**	REST standards followed CRUD operations Proper Standard request/response structure Error handling & Validation	10 – All criteria fully implemented with proper REST practices, complete CRUD, consistent structures, and robust validations. 7 – Mostly implemented; minor deviations or

			small missing elements in one of the criteria. 4 – Partially implemented; noticeable gaps in multiple areas such as structure, REST conventions, or validation. 0 – Not implemented properly; major issues across all criteria.
	Backend Architecture – **10 Marks**	Spring Boot or combination with Node Microservices Layered architecture (Controller → Service → Repository) DTOs/Models used properly Database schema alignment	10 – Clean microservice structure, fully layered architecture, correct DTO/model mapping, and fully aligned database schema. 7 – Mostly correct; minor inconsistencies in layering, DTO usage, or schema mapping. 4 – Partially implemented; noticeable issues in architecture, DTO design, or schema alignment. 0 – Not implemented properly; lacks structure, poor mapping, or incorrect database alignment.
Frontend Development	UI/UX Quality – **10 Marks**	Clean and responsive UI Proper routing Consistent styling	10 – UI is visually clean and fully responsive; routing works flawlessly; styling is uniform; navigation is smooth and intuitive.

(React/Next) – **20 Marks**		Intuitive navigation	7 – Mostly responsive and well-styled; minor routing issues or minor inconsistencies in UI/UX. 4 – Partially responsive; noticeable styling inconsistencies; routing is functional but not well-structured; navigation requires effort. 0 – UI is not clean or responsive; routing is broken or incorrect; styling inconsistent; navigation confusing or non-functional.
	Functional Frontend **10 Marks**	Displays backend data properly Form validations State management (Context/Redux/Local state) Error & success feedback messages	10 – Backend data loads correctly and consistently; strong validation rules; clean and effective state management; clear and user-friendly error/success messages. 7 – Mostly correct implementation; minor gaps in validation, state handling, or message clarity. 4 – Partial implementation; inconsistent data rendering; basic validation; limited or confusing feedback messages.

			0 – Backend data not displayed properly; no real validations; poor state management; missing feedback messages.
Testing & Quality Assurance **10 marks**	Functional Testing **2 Marks**	All core flows tested (Event creation, attendee management, vendor management, budgeting etc.) Positive + Negative test cases Boundary condition handling	2 – Extensive test scenarios & evidence 1 – Partial coverage
	API Testing **3 marks**	Postman/Swagger/OpenAPI collection Correct status codes Error responses Token-based endpoints tested	3 – Comprehensive Postman test suite 2 – Partial API test coverage
	UI Testing **2 Marks**	Form validation tests Navigation flow tests User experience checks Screenshot-based test report	2 – All UI flows tested 1 – Most flows covered
	Unit/Integration Testing **3 Marks**	Unit Test for each method in each layer	3 – Several unit tests + integration tests 2 – At least unit tests present 1 – Only manual tests, no automation
Documentation – **20 Marks**	Project Documentation	Features & module details Backend Endpoints List	5 – All required frontend pages listed with correct URLs; clear screenshots included; repository link shared and accessible.

	Completeness – **5 Marks**	Frontend Pages & URL list Screenshots Source code repository link	3 – Mostly complete; minor issues like missing a few URLs or limited screenshots. 1 – Partially done; few pages listed, minimal screenshots, or incomplete repository link. 0 – Missing pages/URLs, no screenshots, or no repository link provided.
	Diagrams & Design Documentation – **10 Marks**	ER Diagram Wireframes User Flow Document	10 – All diagrams complete & clear 7 – Most diagrams present but missing details 4 – Minimal diagrams 0 – No diagrams
Deployment & DevOps – **10 Marks**	Dockerization – **10 Marks**	Dockerfile for backend Dockerfile for frontend docker-compose setup (bonus consideration) App runs via Docker with minimal steps	10 – Successfully dockerized full app 7 – Docker partially implemented 4 – Only backend OR frontend dockerized 0 – No dockerization
Code Quality & Repository Management – **10 Marks**	Code Cleanliness & Structure – **5 Marks**	Naming conventions Proper folder structure No hardcoded secrets Reusable components & services	10 – Clean and consistent naming; well-organized folders; no secrets in code; reusable components/services; public repo available; clear commit history; well-written README; overall strong project structure.

	GitHub/GitLab Repository – **5 Marks**	Public repository link Meaningful commit messages README file Proper project structure	7 – Mostly correct structure and conventions; minor issues like slightly inconsistent naming, less detailed README, or few weak commit messages. 4 – Partially implemented; folder structure or naming inconsistent; limited reusability; README basic; commit messages unclear. 0 – Disorganized structure; hardcoded secrets; missing README; poor commit practices; no meaningful components or repo link.

Optional (Advanced Skills):

- Configuring your application for using Secrets manager service (Hashi Corp Vault),
- Application health monitoring using Prometheus and Grafana
- Asynchronous Messaging using Kafka
- Deployment on any cloud services (EC2/ECS/ECR/AKS/EKS/Fargate)
- CI/CD Workflows(GitHub Actions/GitLabCI) or Jenkinsfile and IaC (Terraform) for deploying your application
- Usage of any External Services
- Any other pattern implementations

Note:

Implementing the above optional items are not considered in the rubrics due to time and implementation limitations, but are recommended as they were covered during the sessions and you would have a solid understanding of the concepts in building a fully functional secure application with best practices.

It will also help you in having a project with a complete application which can be showcased if needed making you standout.

If these are incorporated in the app, please do mention the same in READMEs or document with necessary information or screenshots.